NPLify · P0 Technical Baseline

Deliverable 9 — Cutover Approach

New XP Technologies Limited

11 October 2026 · Confidential

NPL runs on spreadsheets today. The plan is to start using NPLify on a staging instance: after every change to a spreadsheet the team makes the same change in the app, for three to four weeks, to see that things go in the right direction; earlier deals are entered as well so the history is in one place and the ledger's balances are built from real entries; when everything is right the database is copied to production and the production app is used from a quiet day, chosen a couple of days before. This document gives that plan its steps, its checks and its sign-off.

---

1 · The approach in one page

Three things have to be true on the evening of the switch: the configuration in the app describes NPL's business as it is (Configuration Schema), the ledger's balances equal the spreadsheets' balances to the cent, and the team has already run real weeks in the app and trusts it. The plan gets there without touching production until the last day:

1. **Prepare.** Load the configuration (clients, projects, partners, pairs, fee structures, parties, accounts, templates, users and roles) into the staging instance; validate it against the schema rules; fix the spreadsheets' inconsistencies that the validation surfaces before going further. Pick the start date for the backfill (Section 4).
2. **Run in parallel on staging.** For three to four weeks the spreadsheets stay the system of record and the app is run beside them: every change made to a sheet is made in the app the same day, by the same person, through the normal screens and approvals. Finance reconciles every evening (Section 5); every difference goes in the discrepancy log and is explained before the next day starts.
3. **Backfill alongside.** Deals from the start date up to the first day of the run are entered through the same screens, in date order, so that the ledger holds every posting behind today's balances (Section 4). When the backfill is complete, the app's balances at every partner equal the sheets'.
4. **Go / no-go.** The exit criteria of Section 6 are met or the run is extended; Management signs; NPL names the switch day: the day with the least traffic, a couple of days ahead.
5. **The switch day.** Freeze the sheets after the previous day's close; final reconciliation; reset production once, copy the staging database and the file bucket to it, verify, point the users at production. The next day is the first live day; the sheets stay read-only for four weeks as the fallback; the staging instance is removed.

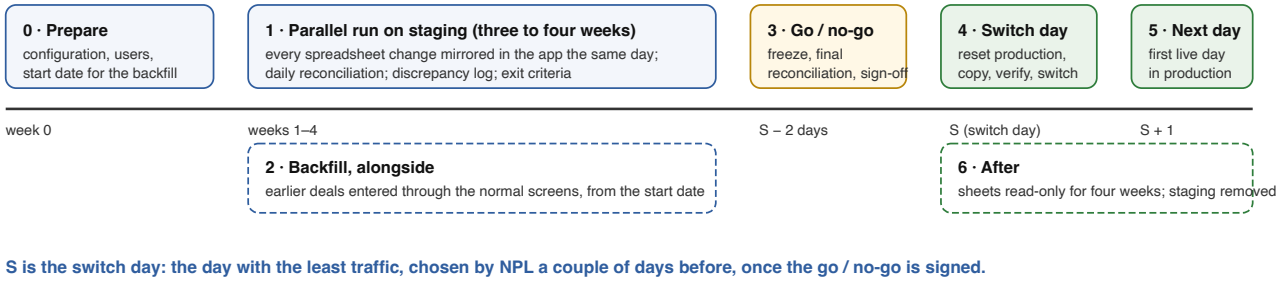


Figure 1 — The cutover in six steps. Only steps 3 and 4 touch production; everything before them is on the staging instance and reversible.

What the parallel run proves, and what it does not. It proves that the configuration prices and routes NPL's real deals the way NPL does, that the postings of each day reproduce the sheets' balances, and that the team can run its day in the app. It does not prove the ledger's arithmetic — that is the Calculation Specification's acceptance vectors and the Ledger Posting Design's worked transactions, which the build already passes — and it does not test every pattern of the Scenario Catalogue: the weeks will bring whichever deals they bring. Patterns that do not occur in the run are exercised once each on staging as scripted test deals before the go/no-go (Section 6).

2 · Starting point: the spreadsheets

NPL's workbooks are the source of every figure the cutover loads, and the yardstick of every check. Two kinds of content are taken from them:

In the sheets	Becomes in NPLify	Notes
Master data	CLIENT, PROJECT, PARTNER (with cutoffs and holidays), CURRENCY_PAIR and rate chains, FEE_STRUCTURE and FEE_TIER per partner × pair, FEE_OVERRIDE, SENDER with allow-lists, RECEIVER_GROUP, RECEIVER, RECEIVING_ENTITY and accounts, PARTNER_ENTITY and endpoints, DESTINATION_APPROVAL, MARKUP_SHARE and PARTNER_REBATE rules, MESSAGE_TEMPLATE, SYSTEM_SETTING, USER / ROLE / ROLE_RIGHT	The Project Context and Payment methods tabs, the fee table and NPL's answers of 8 October are the sources; the Configuration Schema's rules are the validation.
Deals	Deals from the start date to the first day of the run: entered through the normal screens as the backfill (Section 4). Deals from the first day of the run on: entered day by day during the parallel run. Deals closed before the start date: not entered; the sheets remain their record.	The sheet's deal reference becomes the deal's client reference, so a row in the sheet and a deal in the app are the same thing by name.

The validation will surface things the sheets never had to state: a receiver with no approved destination at a partner, a sender with no allow-list, a fee structure whose parts do not add up, a partner with a bank cutoff but no time zone, a pair with no rate source. Each is resolved with NPL before the run — the run must start from a configuration that passes.

3 · The staging instance and the rules of the parallel run

Staging is a second copy of the production stack in NPL's AWS account (Architecture & Stack Note, Environments): the same image, its own database and bucket, its own sign-in. It exists for the weeks of the run and is removed after go-live. Four rules make the run meaningful:

1. **The sheets lead, the app follows — the same day.** Whoever changes a sheet makes the same change in the app before the end of their shift, as the person they are (Operations enters, Finance approves, Management escalates), through the normal screens. No bulk entry, no "Finance types everything": the point is that the roles and approvals are exercised.
2. **Real values, real timestamps.** The partner's rate as quoted, the market rate at the time (the app captures its own; the sheet's rate is a check), the real cutoff, the real evidence attached. Where the app's clock would have expired a quote the sheet kept alive, the discrepancy log says so: that is a finding about policy, not an error.
3. **Outside services point at copies.** Rates, hash checks and the authenticator app work as in production. The client journal writes to a *copy* of each client's sheet, never to the client's real sheet, until go-live. No message rendered on staging is sent to a sender or receiver; quote packages are rendered and filed only.
4. **Finance reconciles every evening** with the checks of Section 5, before the next day's first deal. A difference is either fixed in the app (an entry missed or mis-entered), fixed in the sheet (the sheet was wrong — these are findings NPL wants), or explained as a known difference of method (rounding at the pair's unit, a fee part the sheet computes differently) and recorded. A difference that cannot be explained stops the run until it is.

> **Two things the staging weeks are not.** They are not training on test data — that happens on the local environment before the run starts, with the Ledger Posting Design's worked transactions as the exercises. And they are not optional for Finance: the backfill sign-off and the daily reconciliation are Finance's work, and the go/no-go is signed on Finance's log.

4 · Backfill: earlier deals as real postings

The ledger is append-only and every balance is a sum over postings, so the balances the sheets show today must themselves come from postings. Rather than opening the books with balances typed in, the backfill **enters the deals behind those balances**: NPL picks a start date, and every deal from that date to the first day of the run is entered through the normal screens, in date order, with the sheet's figures — the partner's rate as quoted and the market rate of the day entered as dated rows, the sender rate, fee parts and tier, the partner entity and endpoint, the collection parts with the evidence the sheet linked, the settlement lines and confirmations as they happened. Each posts as it would have on the day, so the ledger holds the history and the balances at the same time, and every figure on a screen can be traced to the deal that produced it.

- **The start date.** Early enough that every balance open today — money at a partner waiting to be settled, collected and not yet converted, NPL's pool at a partner, sender credits, shortfalls, accrued rebates and shares, open loss recoveries, open invoices in NPL-GR — originates on or after it. In practice: the start of the current calendar year, or each project's start if later; a project whose balances all cleared recently can start later. NPL decides per project.
- **Who and when.** Operations enters, Finance approves, as on the day — the same roles, so the approvals are exercised too; in batches per project, alongside the parallel run, oldest first. Each batch ends with the check that the app's balances for that project equal the sheet's at the batch's last date.
- **Dates.** A backfilled deal carries the real dates in its fields and evidence; the posting date is the day it is entered. Reports by deal date are therefore right; the audit log shows when the entry was made, which is also right.
- **What is not entered.** Deals closed before the start date, past approvals and audit, and the sheets' period results: the sheets remain the record for those. The app's profit and loss begins at the start date.
- **When it is complete.** Every project's balances in the app equal the sheets' at the last backfilled date, and Finance signs the backfill off. From then on the daily reconciliation of Section 5 keeps them equal.

> **Why real postings rather than typed-in balances.** A typed-in balance has no deal behind it, cannot be traced, and would need a special kind of transaction the ledger otherwise never has. Entering the deals costs more time during the run, but every balance the app shows on the first live day is then explained by its own history, and the team has entered several weeks of NPL's real deals before it goes live.

5 · Reconciliation checks

Run by Finance every evening of the parallel run, at the end of every backfill batch, and once more before go-live against the frozen sheets. Each check names what the app shows, what the sheet shows and the tolerance; a check outside tolerance is a discrepancy-log entry with an owner and a resolution before the next day.

Check	In the app	In the sheets	Tolerance	Owner
Balance at each partner, per currency	PROJECT_BALANCE per project × partner × currency	The sheet's partner balance for the same project and currency	Exact	Finance
Money collected and not yet converted	Σ CL.<PARTNER>.<CCY>.COLLECTED	Open collections on the sheet	Exact	Finance
Owed to receivers, per group	GROUP_ENTITLEMENT	The sheet's outstanding payouts by receiver	Exact	Finance
Open deals and their states	DEAL_GROUP / DEAL lists by state	The sheet's deal register	Same set of references; states agree	Operations

Check	In the app	In the sheets	Tolerance	Owner
Today's deals	Every deal group created today	Every row added to the sheet today	One-to-one, same amounts, same partner	Operations
Sender credits and shortfalls	CL.<SENDER>.CREDIT, CL.<RECEIVER>.SHORTFALL	The sheet's notes column	Exact	Finance
NPL's money at partners and in own wallets	CO.<PARTNER>.POOL, CO.OWN.WALLET, CUSTODY	The sheet's pool and wallet lines; the wallet's on-chain balance	Exact; on-chain within pending transfers	Finance
Margin of the day	EARN_GROSS postings of the day, per deal	The sheet's margin column (or the Calculation Specification's vectors where the sheet has none)	Within the pair's rounding	Finance
Rebates and shares accrued	PARTNER_REBATE_ACCRUAL, MARKUP_SHARE_ACCRUAL	The sheet's monthly rebate and share working	Exact at month end	Finance
Invoices (NPL-GR)	INVOICE_BALANCE	The reseller workbook's open invoices and instalments	Exact	Finance
Ledger rules	The ledger's balance rules as database checks (balanced per currency and per owner; every balance a sum over postings)	—	Zero violations	Engineering
Visibility	A sample of screens and exports per role	Roles & Visibility Matrix	All pass	Engineering with Management

The discrepancy log is a sheet of its own: date, check, project, app figure, sheet figure, cause (app entry · sheet error · method · policy), resolution, who closed it. It is the evidence the go/no-go is signed on, and it is kept after go-live.

6 · Exit criteria and the go / no-go

1. **Ten consecutive business days** with every check of Section 5 inside tolerance at the evening reconciliation, and no discrepancy open for more than one day during the run.
2. **Every project NPL runs today** has had at least one deal through the app during the run, or a scripted test deal on staging where the weeks brought none (the dormant Evo products need none).
3. **The ten flows of the Scenario Catalogue** have each been exercised at least once on staging, by a real deal or a scripted one: F1 Cash by token (collection or settlement); F2 USDT collection with wallet screening; F3 Receiver wallet screened before settlement; F4 SWIFT collection; F5 SWIFT settlement; F6 Returned transfer; F7 One-time local deposit account; F8 Local payouts for small amounts; F9 Bank cutoff; F10 Internal transfer inside the partner's platform.
4. **Every role has done its own work** in the app: Operations has quoted, verified, instructed and prepared; Finance has approved, confirmed rate versions, reconciled and recorded a recovery or an offset; Management has approved an escalation, a configuration change and a user change.
5. **The visibility checks** of the Roles & Visibility Matrix pass on staging with NPL's real configuration.
6. **The backfill** is complete and signed off by Finance: every project's balances equal the sheets'.
7. **A restore has been rehearsed:** the staging database restored from a snapshot into a scratch instance, and the ledger checks run there.
8. **Management signs** the go/no-go, on Finance's discrepancy log and the engineering checklist, and names the switch day. A no-go extends the run by whole weeks; nothing is rushed into the switch.

7 · The switch day

The switch day is the day with the least traffic, chosen by NPL a couple of days before once the go/no-go is signed. Nothing below depends on which day of the week it is.

When	Who	What
The day before, close	Operations, Finance	Last entries mirrored; the sheets are frozen (read-only copy made); the app's day is closed with the evening reconciliation.
Switch day, morning	Finance	Final reconciliation of every check against the frozen sheets; the discrepancy log closed.
Switch day, midday	Engineering	Production reset once (the single reset of NPL's architecture page): database dropped and recreated empty, bucket emptied of test runs. Staging put in read-only mode. Database snapshot and bucket copied to production; migrations confirmed at the same schema version; the ledger checks run on production; row counts and balance totals compared with staging — identical, or the copy is repeated.
Switch day, afternoon	Finance, Management	Sign in to production: one screen per role compared with staging; balances per partner, entitlement per group, custody and the exception list match the morning's reconciliation. Client journal switched from the copies to the clients' real sheets. Management records the go-live in the audit log.
Next day	Everyone	First live day in production. The sheets are not written to; whoever reaches for one records the item in the app instead. Finance reconciles that evening against the frozen sheets plus the day's deals, as during the run.
After	Engineering	Staging removed after two clean weeks in production (its last snapshot kept 30 days). The frozen sheets stay read-only for four weeks as the fallback, then are archived.

8 · Rollback

Until the switch day, rollback is nothing: staging is discarded or the run extended. On the switch day, if the copy or the verification fails, production is left empty, staging goes back to read-write and the run continues; a new switch day is chosen. After the first live day, if the app cannot be used for a day, the team works from the frozen sheets plus the app's entries so far (the Architecture note's fallback) and enters the day into the app afterwards with the real timestamps in

the evidence; the ledger is never edited to catch up, it is posted. A restore from backup is the last resort and a Management decision: the ledger is then the ledger as of the restore minute, and the gap is re-entered from the sheets.

9 · Roles during the cutover

Role	During the run	On the switch day
Operations	Mirrors every operational change the same day; enters the backfill in batches; reports what the app would not let them do (a finding either way).	Nothing to do; first live entries the next day.
Finance	Approves as Finance in the app, for the day's deals and the backfill; reconciles every evening and at the end of every batch; owns the discrepancy log; signs the backfill off.	The final reconciliation; the afternoon sign-in check.
Management	Approves escalations and configuration in the app as Management; reads the log weekly.	Signs the go/no-go and names the day; records the go-live.
New XP engineering	Runs staging; loads and validates the configuration; runs the ledger checks nightly; fixes what the run finds, as releases to staging only.	The reset, the copy, the verification, the switch; on call the next day.

10 · Assumptions and decisions for NPL

1. **Three to four weeks** is the planned length of the run; the exit criteria, not the calendar, end it.
2. **Start date of the backfill**: the start of the current calendar year, or each project's start if later, and early enough that every open balance originates after it. NPL decides per project.
3. **The backfill is entered through the screens**, not loaded in bulk, so the postings and the approvals are real. If the volume turns out too large for the run, the start date moves forward for the projects concerned, and Finance confirms that no open balance is left behind.
4. **The one reset of production** is the one on NPL's architecture page; the copy from staging means production starts from the reconciled run's data, then live data after.
5. **Client journals** write to copies during the run and to the real sheets from the switch day.
6. **The switch day** is NPL's choice, a couple of days ahead, on the least traffic; the runbook of Section 7 does not depend on the weekday.
7. **Staging cost**: a second small service and database for the weeks of the run, about the running cost again (Architecture & Stack Note).